MODULE 13 · DOMAIN 2 - CONTAINER ORCHESTRATION

The network model

Why every Pod gets its own IP, who wires it up, and how Service traffic finds a Pod.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 13 of 35

admin@sthithapragna.com

Every Pod gets **its own IP address**, and every Pod can reach every other Pod **without NAT**.

admin@sthithapragna.com

Four networking problems, four answers

PROBLEM	SOLVED BY	MODULE
Container to container, in one Pod	Shared network namespace: localhost	6
Pod to Pod, across nodes	The flat Pod network, built by a CNI plugin	13
Pod to Service	A stable virtual IP, made real by kube-proxy	13, 14
Outside world to Service	NodePort, LoadBalancer, Ingress, Gateway API	14, 15

ON THE EXAM

Keep these four apart, and every networking question has an obvious home.

The rules of the Kubernetes network model

- **Each Pod has a unique IP** across the whole cluster.
- **Pods reach Pods on any node without NAT.** The address a Pod sees for itself is the one others see.
- **Node agents,** such as the kubelet, can reach every Pod on their node.
- **Kubernetes defines the rules; it ships no implementation.** A plugin must provide one.

> ON THE EXAM
>
> No NAT between Pods: an application can hand out its own IP, and peers can use it.

Three address ranges that must never overlap

RANGE	ASSIGNED TO	EXAMPLE
Node network	The nodes themselves	10.0.0.0/16
Pod CIDR	Pods; each node usually gets its own slice	10.244.0.0/16
Service CIDR	Virtual ClusterIPs for Services	10.96.0.0/12

ON THE EXAM

Overlapping ranges cause traffic to vanish in confusing ways. They are chosen when the cluster is built.

CNI: the Container Network Interface

- **A specification and libraries** for plugins that connect containers to a network. A CNCF project.
- **The runtime calls the plugin** when it creates a Pod's network namespace, and again when it deletes it.
- **The plugin** creates the interface, assigns an IP (IPAM), and sets up routes.
- **No plugin, no Pod network:** nodes stay NotReady and Pods cannot start.

> ON THE EXAM
>
> CNI for networks, CSI for storage, CRI for runtimes: three plug-in standards.

What happens to the network when a Pod starts

01
Scheduled
The Pod is bound to a node.

›

02
Sandbox
The kubelet asks the runtime for a Pod sandbox.

›

03
Namespace
The runtime creates the Pod's network namespace.

›

04
CNI ADD
The plugin adds an interface, an IP and routes.

›

05
Containers
App containers start inside that namespace.

ON THE EXAM

The network exists before any app container starts, which is why every container in the Pod shares it.

Network plugins you should recognise

PLUGIN	KNOWN FOR
Calico	Routed networking with BGP, or an overlay; strong NetworkPolicy; optional eBPF data plane
Cilium	eBPF data plane; NetworkPolicy including layer 7; can replace kube-proxy. CNCF graduated
Flannel	A simple overlay network. Does not enforce NetworkPolicy on its own
Cloud VPC plugins	Pods get real addresses from the cloud network, such as the AWS VPC CNI
Multus	A meta-plugin: several network interfaces per Pod

ON THE EXAM

Which plugin you choose decides whether NetworkPolicy is enforced at all.

Overlay versus routed networking

OVERLAY

Tunnel between nodes

- Pod packets wrapped, e.g. in VXLAN
- Works on almost any underlying network
- Small overhead per packet
- Flannel, or Calico in overlay mode

ROUTED

Real routes to Pods

- Pod IPs routed directly, no wrapping
- Needs the network to know Pod routes
- No encapsulation overhead
- Calico with BGP, cloud VPC plugins

THE TIEBREAKER

Both satisfy the model: flat, no NAT between Pods. They differ in how packets cross between nodes.

NetworkPolicy depends on the plugin

- **Kubernetes owns the NetworkPolicy API.** You can always create the objects.
- **The plugin does the enforcing.** A plugin without policy support ignores them.
- **Silently.** No error, no warning: the policies simply do nothing.
- **Check the plugin first** when policies seem to have no effect. Module 19 covers the rules.

> ON THE EXAM
>
> Policies accepted but not enforced: the network plugin does not support NetworkPolicy.

Dual-stack: IPv4 and IPv6 together

- **Stable since Kubernetes 1.23.** Pods and Services can have both an IPv4 and an IPv6 address.
- **Services choose** with ipFamilyPolicy: SingleStack, PreferDualStack or RequireDualStack.
- **Needs support throughout:** the plugin, the nodes, and dual Pod and Service ranges.

> ON THE EXAM
>
> IPv6-only and dual-stack clusters are both possible, if the network plugin supports them.

Pod IPs change whenever Pods are replaced. A **Service** gives a stable address, and **kube-proxy** makes it work.

How a packet to a Service reaches a Pod

01

Send

A client Pod sends to the Service's ClusterIP and port.

02

Match

Rules on the node, written by kube-proxy, match that virtual IP.

03

Choose

One ready backend is picked from the EndpointSlices.

04

Rewrite

The destination becomes that Pod's real IP.

05

Deliver

The Pod network carries it there.

ON THE EXAM

A ClusterIP is virtual. Nothing listens on it; rules on each node rewrite traffic sent to it.

kube-proxy modes

MODE	HOW	STATUS
`iptables`	One set of iptables rules per Service	Default on Linux
`nftables`	The newer nftables framework; scales far better	Stable since 1.33; recommended
`ipvs`	The kernel's IP Virtual Server load balancer	Deprecated in 1.35; to be removed
`No kube-proxy`	The CNI plugin handles Services, e.g. Cilium with eBPF	Plugin-dependent

ON THE EXAM

Old advice said 'use IPVS for big clusters'. Today's answer is nftables.

Currency check: kube-proxy and endpoints

- **IPVS mode is deprecated** since 1.35. It now logs a warning and is on the way out.
- **nftables** is the recommended mode for large clusters. iptables remains the default.
- **The Endpoints API is deprecated** since 1.33, in favour of EndpointSlices.
- Older material says 'switch to IPVS for scale' and shows Endpoints objects. Both are dated.

> ON THE EXAM
>
> Scale problem with kube-proxy today: nftables. Backend lists today: EndpointSlices.

EndpointSlices

- **The list of backends for a Service:** Pod IPs, ports, and whether each is ready.
- **Maintained by the EndpointSlice controller** from the Service's selector and Pod readiness.
- **Split into slices,** up to 100 endpoints each by default, so big Services update cheaply.
- **Readiness decides membership.** A Pod failing its readiness check stops receiving traffic.

> ON THE EXAM
>
> Service with no traffic reaching Pods? Check the EndpointSlice: empty usually means a selector or readiness problem.

Looking at a Service's backends

```
$ kubectl get endpointslices -l kubernetes.io/service-name=booking-api
NAME                ADDRESSTYPE   PORTS   ENDPOINTS                          AGE
booking-api-7x2kq   IPv4          8080    10.244.1.14,10.244.2.9,10.244.3.21   3d
```

1 Slices are linked to their Service by a standard label.

3 Three ready Pod IPs, on three different nodes' Pod ranges.

THE TAKEAWAY

If ENDPOINTS shows none, compare the Service selector with the Pods' labels, then check readiness.

> In a Quayside test cluster, Pods can reach other Pods on the same node by IP. Pods on different nodes cannot reach each other at all.
>
> The test uses Pod IPs directly, not Services or DNS names. No NetworkPolicies exist.

Where is the fault most likely to be?

A The Pod network is not routing between nodes

B kube-proxy has stopped programming Service rules

C CoreDNS is failing to resolve the Pods' names

D A default-deny NetworkPolicy is blocking traffic

◆ PAUSE AND DECIDE

Where is the fault most likely to be?

A The Pod network is not routing between nodes

Same node works, cross-node fails: the plugin's overlay or routes.

B kube-proxy has stopped programming Service rules

kube-proxy handles Service IPs. This test uses Pod IPs.

C CoreDNS is failing to resolve the Pods' names

No names are used; the test goes straight to IPs.

D A default-deny NetworkPolicy is blocking traffic

None exist, and one would block same-node traffic too.

THE SIGNAL

Pod IP to Pod IP, failing only across nodes: the CNI plugin's cross-node path.

> Quayside creates NetworkPolicies so only the booking API can reach the bookings database. kubectl shows the policies created without error.
>
> Yet every Pod can still connect to the database. The cluster uses Flannel as its only network plugin.

What is the most likely reason?

A NetworkPolicies only apply to namespaces labelled for them

B kube-proxy enforces policies and must use nftables mode

C New policies take effect after the Pods are next restarted

D The network plugin in use does not enforce NetworkPolicy

◆ PAUSE AND DECIDE

What is the most likely reason?

A NetworkPolicies only apply to namespaces labelled for them

No such label is needed. Policies select Pods directly.

B kube-proxy enforces policies and must use nftables mode

kube-proxy handles Services, not policy.

C New policies take effect after the Pods are next restarted

Enforcing plugins apply policies to running Pods.

D The network plugin in use does not enforce NetworkPolicy

Flannel alone accepts the objects but enforces nothing.

THE SIGNAL

Policies created fine but ignored: check whether the plugin enforces them.

> Quayside's largest cluster runs Kubernetes 1.37 with about 20,000 Services. kube-proxy runs in IPVS mode, and its logs warn that the mode is deprecated.
>
> The team wants a supported mode that stays fast with this many Services.

Which kube-proxy mode should they move to?

A Stay on IPVS and ignore the warning

B nftables mode

C The userspace proxy mode

D iptables mode, the default

◆ PAUSE AND DECIDE

Which kube-proxy mode should they move to?

A Stay on IPVS and ignore the warning

Deprecated since 1.35 and due for removal.

B nftables mode

Stable since 1.33, and built to scale where iptables struggles.

C The userspace proxy mode

Removed from kube-proxy years ago.

D iptables mode, the default

Supported, but slows down with tens of thousands of Services.

THE SIGNAL

'Supported and fast at scale' today means nftables.

> The booking API Service selects 3 Pods. One Pod starts failing its readiness check, but keeps running and does not restart.
>
> Clients keep calling the Service's ClusterIP throughout.

What happens to the traffic?

A It is still spread evenly across all three Pods

B The Service stops routing until all Pods are ready

C The unready Pod gets none; the other two share it

D kube-proxy restarts the failing Pod to recover it

◆ PAUSE AND DECIDE

What happens to the traffic?

A It is still spread evenly across all three Pods

Readiness exists precisely to stop this.

B The Service stops routing until all Pods are ready

Ready Pods keep serving; one bad Pod does not stop a Service.

C The unready Pod gets none; the other two share it

It is marked not ready in the EndpointSlice, so no traffic.

D kube-proxy restarts the failing Pod to recover it

kube-proxy never restarts Pods; liveness probes cause restarts.

THE SIGNAL

Readiness removes a Pod from traffic without restarting it.

Module 13 in five lines

- **Every Pod its own IP; Pod to Pod without NAT.** Kubernetes sets the rules, plugins implement them.
- **CNI**: the runtime calls the plugin to add an interface, an IP and routes.
- **NetworkPolicy** is only enforced if the plugin supports it. Flannel alone does not.
- **kube-proxy** makes ClusterIPs work: iptables default, **nftables** recommended, IPVS deprecated.
- **EndpointSlices** list ready backends; readiness decides who gets traffic.

We have seen how a ClusterIP works. Next: **every type of Service**, and how DNS names them.